IT305 LAB · SOCKET PROGRAMMING PROJECT

Fault Tolerant Topic-based File Distribution Service

Design Document

Project group	**021_034**
Lab groups	021 and 034 (merged)
Submitted	27 September 2026
Platform	C11 on Linux, POSIX sockets (IPv4, TCP), POSIX threads, GNU Make

Group members

Name	Roll number	Lab group	Planned responsibility
Aaditya Kaushik	202401001	034	Part I server: topic map, single and multi-threaded modes
Nisarg Trivedi	202401129	034	Part I client, buffered line reader, metrics logging
Niranjan Dangi	202401042	034	Fault injection and Part II Case 1 retry logic
Kavish Inani	202401089	021	Part II Case 2: session table and resume protocol
Neel Khatri	202401093	021	Case 2 Enhanced: parallel connections and optional optimizations
Manal Patel	202401112	021	Experiment scripts, plots, report integration

Contents

1 System overview 2
2 Server data structures: topic to directory map 3
3 Client-server message formats 4
4 Fault-injection mechanism 6
5 Session management protocol (Case 2) 7
6 Performance enhancements (Case 2 Enhanced) 9
7 Additional features beyond the minimum 11
8 Code structure 12
9 Measurement methodology and experiment plan 14
10 Design thought process and alternatives 15
References 17

1 System overview

The service is one C codebase built in four layers: a topic file server (Part I), then fault injection with restart (Case 1), then resumable sessions (Case 2), then parallel connections (Case 2 Enhanced). Each layer reuses the code of the one before it, so any behaviour difference between two cases comes from exactly one added mechanism.

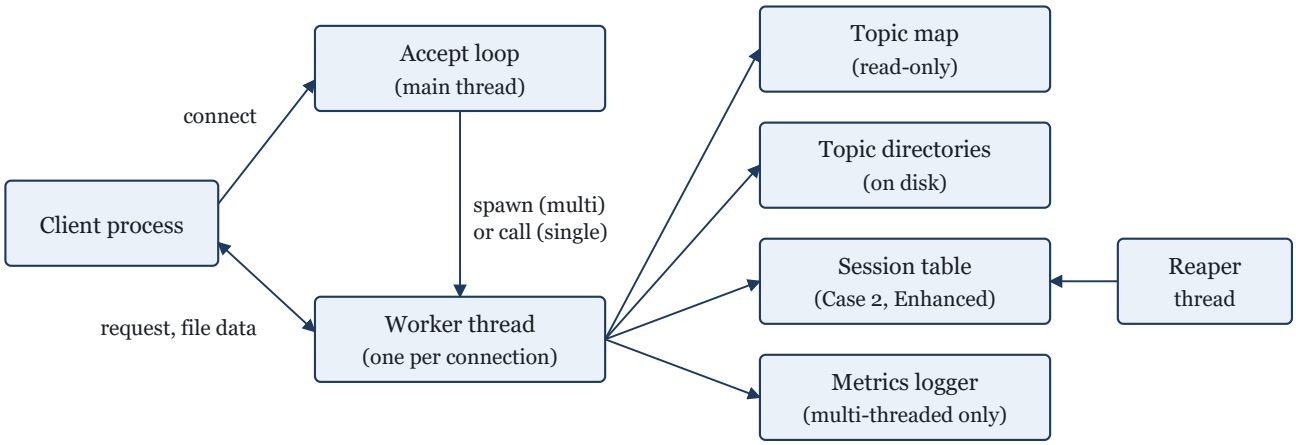


Figure 1. Server structure. The main thread only accepts connections and worker threads do all per-client I/O. The topic map is built once at startup and never written again. The session table is the only shared mutable state.

In Part I single-threaded mode the server process has exactly one thread. That thread accepts a client, serves it completely, and only then accepts the next one. No logger or reaper thread is created in this mode: the serving thread writes each metrics row itself after the client's end time has been taken, so logging never falls inside a measured interval. The logger thread exists only in the multi-threaded servers, and the reaper only in Case 2 and Case 2 Enhanced.

Table 1. How the four submission folders differ.

Folder	Server threads	Fault injection	Recovery after a cut	Connections per topic
`Part1/`	`mode=single`: one thread serves all clients in turn. `mode=multi` (default): one thread per client	None	None	1
`Part2_Case1/`	One thread per client	Yes, probability p	Restart the topic from byte 0	1
`Part2_Case2/`	One thread per client, plus session reaper	Yes, probability p	Resume from the client's checkpoint	1
`Part2_Case2_Enhanced/`	One thread per connection, plus session reaper	Yes, per connection	Only the cut connection's batch resumes; the others continue	N (default 4)

2 Server data structures: topic to directory map

The server scans `<topics_root_dir>` once at startup and builds a sorted array of topics, each holding a sorted array of its files. Lookups use `bsearch()`. Nothing is hard-coded: every sub-folder of the root becomes a topic whose key is the folder name.

```
typedef struct {
    char     name[256];      /* file name inside the topic directory */
    uint64_t size;           /* bytes, from stat() at startup */
    uint64_t pos;            /* payload position: sum of the sizes of all earlier files */
} file_entry_t;

typedef struct {
    char          key[64];         /* topic = sub-folder name, e.g. "cane" */
    char          path[PATH_MAX];  /* absolute path from realpath() */
    file_entry_t *files;           /* sorted by name, so every file has a stable index */
    uint32_t      file_count;
    uint64_t      total_bytes;     /* sum of file sizes (the useful bytes) */
} topic_t;

typedef struct {
    topic_t  *topics;              /* sorted by key; never modified after startup */
    uint32_t  count;
} topic_map_t;
```

With the Animals-10 dataset the map resolves as below. The root is whatever path is passed on the command line.

Table 2. Topic map for the Animals-10 dataset.

Key (topic)	Value (directory path)
cane	`<topics_root_dir>/cane`
cavallo	`<topics_root_dir>/cavallo`
elefante	`<topics_root_dir>/elefante`
farfalla	`<topics_root_dir>/farfalla`
gallina	`<topics_root_dir>/gallina`
gatto	`<topics_root_dir>/gatto`
mucca	`<topics_root_dir>/mucca`
pecora	`<topics_root_dir>/pecora`
ragno	`<topics_root_dir>/ragno`
scoiattolo	`<topics_root_dir>/scoiattolo`

Design notes

- The map is never modified after startup, so worker threads read it with no lock at all and it adds no contention in the multi-threaded server. A topic pointer held by a session can never be freed or changed underneath it.
- Files are sorted by name, so each file has a stable index from 0 to $n - 1$, where n is the file count. Checkpoints and parallel batches refer to files by this index.

- The precomputed `pos` field turns a checkpoint (file index, offset) into a single byte position in the topic (Section 5.4), and lets the server compute the size R of any response in constant time for the fault injector.
- Topic directories are assumed not to change while the server runs. A new topic is simply a new folder, picked up at the next server start.

Lookup rules

- Exact match first, then case-insensitive (`strcasecmp`), so `GET Cane` and `GET cane` both work.
- Keys that are empty, longer than 63 bytes, or contain `/` or `..` are rejected with `ERR 400` .
- Only regular files (`S_ISREG`) are listed. Hidden files, sub-directories and files whose names contain a newline are skipped, so every file header fits on one line.
- With 10 topics, `bsearch()` needs at most $\lceil \log_2 10 \rceil = 4$ comparisons, so a hash table would add code without a measurable gain. The lookup sits behind `topic_lookup()` so it can be replaced later.

3 Client-server message formats

All control messages are single ASCII lines ending in `\n` . File contents follow their header as raw bytes whose exact length the header states. Text headers keep the required `GET <topic>` form and are easy to debug with `nc` and Wireshark. A file header is around 50 bytes, well under 1% of an image of a few tens of kilobytes.

Common rules for every case

- Fields are separated by one space, numbers are unsigned decimal, and a line is at most 1024 bytes including the newline.
- In lines that carry a file name, the name is the last field and runs to the end of the line, so names containing spaces work.
- Errors are `ERR <code> <REASON>` , after which the server closes the connection.
- The client reads lines with a buffered line reader (`lr_readline`) and file data with `lr_read_exact` , which drains the line buffer first, so a header and its data may arrive in the same TCP segment.

3.1 Part I

```
C: GET cane
S: OK 4863 <total_bytes>
S: FILE <size> <name>
S: <size raw bytes>
   ... one FILE block per file, 4863 in all ...
S: END 4863 <total_bytes>
```

Table 3. Part I messages.

Direction	Message	Meaning
C → S	`GET <topic>`	Request every file of a topic
S → C	`OK <file_count> <total_bytes>`	Topic found; totals announced
S → C	`ERR 404 NO_SUCH_TOPIC` or `ERR 400 BAD_REQUEST`	Request rejected
S → C	`FILE <size> <name>` , then `<size>` raw bytes	One file; repeated `file_count` times
S → C	`END <file_count> <total_bytes>`	Transfer complete; server closes

The client creates `<output_dir>/<topic>/` using the topic exactly as given on its command line, writes each file there, and exits with code 0 only after `END` arrives with totals matching `OK`. End of stream before `END` is a failure (exit code 2).

3.2 Part II Case 1 (no session management)

The wire format is identical to Part I and no message carries state. The differences are only in behaviour:

- The server may cut the connection at any byte of the response (Section 4).
- The client treats end of stream or `ECONNRESET` before `END` as a failure. It deletes the files it wrote in that attempt, waits (backoff starting at 100 ms and doubling up to 2 s), reconnects and sends the same `GET <topic>` again, up to `max_retries` reconnections (default 50). An `ERR` reply is not retried.
- Case 2 uses the same backoff and `max_retries`, so completion times of the two cases differ only because of what each retry has to resend.

3.3 Part II Case 2 (session management)

Table 4. Case 2 messages.

Direction	Message	Meaning
C → S	`GET <topic>`	Start a new session (same form as Part I)
S → C	`SESSION <sid> <file_count> <total_bytes>`	New session id (32 hex characters) and topic totals
C → S	`RESUME <sid> <topic> <file_index> <offset>`	Continue from the client's last checkpoint
S → C	`RESUMED <file_index> <offset> <remaining_bytes>`	Checkpoint accepted; data follows
S → C	`ERR 410 SESSION_EXPIRED`, `ERR 409 TOPIC_MISMATCH`, `ERR 416 BAD_CHECKPOINT`	Resume refused; the client starts a fresh `GET`
S → C	`FILE <index> <size> <start> <name>`, then `<size> - <start>` raw bytes	One file, or the rest of a partly received file
S → C	`END <file_count> <total_bytes>`	All files sent
C → S	`DONE <sid>`	Client has everything on disk
S → C	`BYE`	Server has deleted the session

- `<start>` is 0 for every file except the first one after a `RESUMED`, where it equals the checkpoint offset. `<remaining_bytes>` counts file data only.
- A checkpoint of `file_index` equal to `file_count` with offset 0 means every file is on disk and only `END` was lost. The server answers `RESUMED <file_count> 0 0` followed immediately by `END`.
- The client treats `END` as completion. `DONE` and `BYE` are best-effort cleanup: if the connection drops before `BYE`, the transfer still counts as complete and the reaper removes the session after its timeout. Fault injection covers the response up to and including `END`, never the `DONE` / `BYE` exchange.
- If a cut happens before the whole `SESSION` line arrives, the client has no session id yet and simply retries `GET`. The orphaned session is removed by the reaper.

3.4 Part II Case 2 Enhanced (parallel connections)

Connection 1 opens the session and receives a manifest. Every connection, including extra ones and reconnections, then pulls batches of files with `FETCH`.

Table 5. Case 2 Enhanced messages.

Direction	Message	Meaning
C → S (conn. 1)	`GET <topic> <n_conns>`	New parallel session with N = `n_conns` connections
S → C	`SESSION <sid> <file_count> <total_bytes>`	As in Case 2
S → C	`M <index> <size> <name>`, one per file, then `MEND`	Manifest, so the client can plan batches
C → S (conn. 2 to N, or after a cut)	`ATTACH <sid>`	Join an existing session
S → C	`ATTACHED <sid>` or `ERR 410 SESSION_EXPIRED`	Join accepted or refused
C → S	`FETCH <sid> <first_index> <last_index> <offset>`	Send files `first` to `last`, starting `offset` bytes into the first
S → C	`FILE <index> <size> <start> <name>` and data, repeated	Same format as Case 2
S → C	`BEND <first_index> <last_index>`	Batch finished; the connection stays open for the next `FETCH`
C → S (one conn.)	`DONE <sid>`	All batches complete
S → C	`BYE`	Session deleted

After `MEND`, the `GET` connection behaves like any attached connection and sends `FETCH` requests too. The server uses `n_conns` when choosing fault points (Section 6.1). A cut during the manifest is handled like a cut before `SESSION` in Case 2: the client retries `GET`.

4 Fault-injection mechanism

Definition of p. For each transfer request the server receives (`GET` or `RESUME`; in Case 2 Enhanced, each new connection, see Section 6.1), it performs exactly one Bernoulli trial with probability p, the `<failure probability>` argument. If the trial selects a failure, one cut point K is drawn uniformly over the R response bytes the server is about to send, and the connection is aborted when K bytes have been sent. So p is the probability that one request experiences one injected failure. It is not a per-byte or per-chunk rate. Every reconnection is a new request with its own independent trial.

When and how the decision is made

1. The worker thread parses the request and computes R, the number of response bytes it is about to send (headers and file data), from the precomputed sizes in the topic map.
2. It draws $u \sim \mathcal{U}[0, 1)$ from its own generator: `erand48()` with per-thread state seeded from `seed` XOR the connection id. The seed defaults to the start time and can be fixed with the optional argument `seed=<n>` for repeatable runs.
3. If $u < p$, it draws $K \sim \mathcal{U}\{0, 1, \ldots, R-1\}$. Otherwise $K = R$ and there is no cut.
4. The send path caps every `send()` or `sendfile()` so the byte counter never passes K. When the counter reaches K (immediately, if $K = 0$), the thread sets `SO_LINGER` to on with a 0 s timeout and calls `close()`. This sends a TCP RST, which is what a client sees when a server crashes or a link drops.

5. The thread logs `FAULT conn=<id> topic=<t> cut_at=<K> of=<R>` and exits normally. In Case 2 the session record stays in the table for a resume.

Because K ranges over the whole response, a cut can land before the first header, inside a header line, inside file data or between two files, so every recovery path is exercised. Only the server injects faults. `SIGPIPE` is ignored and sends use `MSG_NOSIGNAL`, so neither side is killed by a write to a dead socket.

Edge values. $p = 0$ never cuts. At $p = 1$ every connection is cut before its last byte, because $K \leq R - 1$, so no connection ever delivers `END` and neither case can complete. Case 1 restarts from byte 0 each time. Case 2 still moves its checkpoint forward whenever a cut falls after a new checkpoint, so it can receive almost all of the file data, but it cannot finish. Both clients stop after `max_retries` with exit code 2. Our experiments therefore use $p \leq 0.9$.

Why per request rather than per chunk

A per-chunk coin flip with probability q on a topic of S bytes sent in 64 KiB chunks gives

$$P(\text{complete}) = (1 - q)^n, \qquad n = \left\lceil \frac{S}{64\ \text{KiB}} \right\rceil.$$

For a 100 MiB topic, $n = 1600$, and even $q = 0.01$ gives $0.99^{1600} \approx 1.0 \times 10^{-7}$, so Case 1 would practically never finish. The per-request model keeps p meaningful and gives closed-form predictions. With S the full response size of a topic, the number of failed attempts is geometric with mean $p/(1 - p)$, and each failed attempt sends K bytes with mean $(S - 1)/2$. For Case 1:

$$\mathbb{E}[\text{attempts}] = \frac{1}{1 - p}, \qquad \mathbb{E}[\text{bytes sent in failed attempts}] = \frac{p}{1 - p} \cdot \frac{S - 1}{2} \approx \frac{p}{1 - p} \cdot \frac{S}{2}.$$

The first formula also holds for Case 2, because only an uncut connection can deliver `END` and every attempt is a fresh trial. Session management reduces the bytes each retry costs, not the number of retries. With a checkpoint interval of c bytes (default 64 KiB), each cut discards fewer than c received payload bytes, so for Case 2

$$\mathbb{E}[\text{redundant bytes}] < \frac{p}{1 - p} \cdot c.$$

Checking these predictions against measurements is part of our Part II analysis. A per-chunk mode stays available as the optional argument `fault_mode=chunk` for extra experiments.

5 Session management protocol (Case 2)

The client's checkpoint is the authority on progress, because only the client knows which bytes reached its disk. The server keeps a session record to validate that checkpoint and to account for bytes sent.

5.1 Message sequence

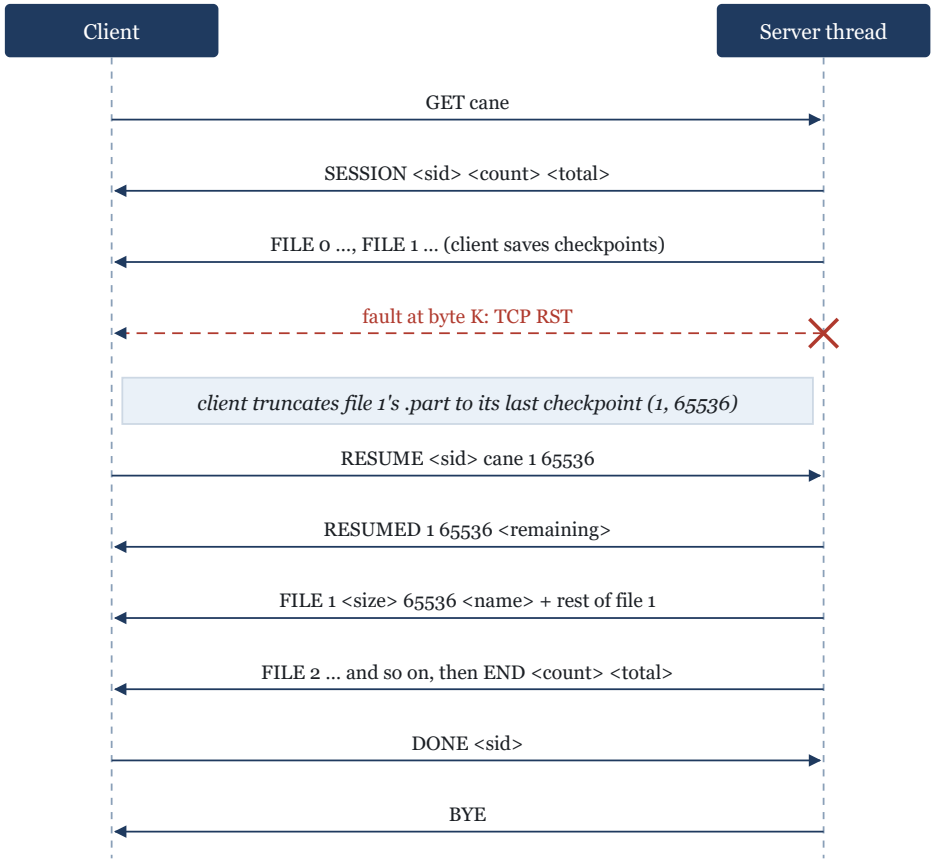


Figure 2. A Case 2 transfer interrupted inside file 1, after the client checkpointed the first 64 KiB of that file.

A refused `RESUME` (`ERR 409`, `410` or `416`) makes the client delete its partial files and checkpoint and start again with `GET`, so the transfer still completes, only without the saving.

5.2 State stored by the client

Table 6. Client-side state in Case 2.

Item	Where	Updated
Checkpoint: `v1 <sid> <topic> <file_index> <offset> <bytes_done>`	`<output_dir>/<topic>/.ckpt`	After every completed file (recorded as the next file at offset 0) and every c = 64 KiB within a file (`ckpt_kb=`)
Partial file	`<output_dir>/<topic>/<name>.part`	Continuously; renamed to `<name>` when complete
Attempt counters (attempts, bytes received, redundant bytes)	Memory, then the client CSV	Every attempt

The write order makes sure a checkpoint never claims data that is not in the file. Data is written and `fflush()`ed first. The checkpoint is then written to `.ckpt.tmp` and `rename()`d over `.ckpt`, which is atomic on POSIX. On reconnect the client truncates the `.part` file to the checkpoint offset, discarding bytes received after the last checkpoint; those are counted as redundant. Because the checkpoint is on disk, a client process that is killed and restarted also resumes (we do not try to survive a power failure). `.ckpt` is deleted when `END` is received, leaving only the topic's files in the output directory.

5.3 State stored by the server

```
typedef struct session {
    char            sid[33];      /* 128 random bits, hex, from /dev/urandom */
    const topic_t  *topic;
    pthread_mutex_t mu;           /* guards every field below */
    uint64_t        hw_pos;       /* high-water mark: payload position of the
                                     furthest byte handed to the kernel */
    uint64_t        bytes_sent;   /* all attempts, for overhead metrics */
    uint32_t        attempts;
    uint32_t        generation;   /* bumped on each RESUME */
    uint32_t        refs;         /* live connections (Case 2 Enhanced) */
    time_t          last_seen;    /* updated on every request and every send */
    struct session *next;         /* hash-bucket chain */
} session_t;
```

Sessions live in a 256-bucket hash table keyed by `sid`. One table mutex guards only insertion, lookup and removal. Each session has its own mutex for its progress fields. The serving thread updates `hw_pos` after every `send()` or `sendfile()` call returns, not only at file boundaries, so the mark keeps pace with the client's in-file checkpoints. In Case 2 only one thread serves a session at a time, so this lock is uncontended. After an abort the mark can overstate what the client received, because the RST discards data still in the send buffer. That is safe: the mark is an upper bound used only to reject impossible checkpoints.

A reaper thread removes sessions that have no live connection and have been idle for more than 300 s (optional argument `session_ttl=<s>`). This is far longer than a client's longest retry sequence, which is under 100 s with the default backoff and `max_retries`.

5.4 Validation on RESUME

Let n be the topic's file count and s_j the size of file j. A checkpoint (i, o) is converted to a payload position

$$\mathrm{pos}(i, o) = \sum_{j<i} s_j + o,$$

which is the `pos` field of file i plus o (for $i = n$, it is the topic's `total_bytes`). Positions are compared rather than (index, offset) pairs because the end of file i and the start of file $i + 1$ are the same point in the stream. The server accepts `RESUME <sid> <topic> <file_index> <offset>` only if all of the following hold:

1. The `sid` exists and has not expired (else `ERR 410`).
2. The topic matches the session's topic (else `ERR 409`).
3. Either $i < n$ and $o \le s_i$, or $i = n$ and $o = 0$; and $\mathrm{pos}(i, o) \le$ `hw_pos` (else `ERR 416`).

The server then increments `generation`. If an older thread for the same session is still blocked on a dead socket, it sees the changed generation at its next chunk and exits without touching the session again, so two threads never send the same session.

6 Performance enhancements (Case 2 Enhanced)

The mandatory enhancement, which we will implement and evaluate in full, is N parallel TCP connections per topic (default 4, optional client argument `conns=<n>`) fed by a dynamic batch queue. The optimizations in Section 6.2 are optional. Each is added only after the parallel version passes `verify.sh` at every tested p, and each can be switched off for the ablation.

6.1 Parallel connections: work split, reassembly, recovery

Table 7. Design of the parallel transfer.

Aspect	Design
Split	From the manifest, the client cuts the sorted file list into batches of consecutive files of about 1 MiB each (at least one file). All batches go into one mutex-protected queue.
Scheduling	N worker threads each own one connection. A worker takes the next batch, sends `FETCH`, receives the batch and takes another on the same connection. Fast connections simply take more batches, so load balances without a static plan.
Reassembly	Every file is written by exactly one worker to its own `.part` file, located by file name, so no byte-level reordering is needed. Files are renamed when complete.
Checkpoint	`.ckpt` holds the `sid`, a completed-file bitmap (one bit per file) and the (index, offset) of each in-progress batch. It is rewritten under the queue mutex after each file and every 64 KiB, using the same write order as Case 2.
Recovery	When a connection is cut, only its batch is affected. The worker truncates its current file to the last checkpoint, puts the batch back at the front of the queue starting from that point, reconnects with `ATTACH` and continues. The other $N - 1$ connections keep transferring during the reconnect. If `ATTACH` is refused with `ERR 410`, the client opens a new session with `GET`, keeps the files already complete, and fetches only the rest.
Completion	A worker closes its connection when the queue is empty. When every file is complete, the last worker sends `DONE`. As in Case 2, `BYE` is best-effort.
Server side	Each connection is served by its own thread. The connections share one session record, reference-counted through `refs` and protected by the session mutex. A `FETCH` is validated by `sid` and bounds: first index ≤ last index < n, and offset ≤ the size of the first file. There is no high-water check, since batches are not contiguous.
Fault injection	Decided once per TCP connection (at `GET` or `ATTACH`), not per `FETCH`; see below.

Why faults are per connection here. A trial per `FETCH` would give a topic of S bytes about $S/1$ MiB chances to be cut, against one chance per connection in Case 2, so Case 2 Enhanced would face far more faults than Case 2 at the same p and the comparison would be unfair. Instead, each connection gets one trial with probability p. If it selects a failure, the cut point is drawn over that connection's expected share of the remaining work:

$$K \sim \mathcal{U}\left\{0, 1, \ldots, \left\lceil \frac{R_{\text{rem}}}{N} \right\rceil - 1\right\},$$

where R_{rem} is the server's estimate of the bytes still to send (total bytes minus the batches already finished with `BEND`). The connection is cut when its byte counter reaches K, or not at all if it runs out of work first. A reconnection is a new trial. Since N connections receive N trials, the results report the number of injected faults per topic next to p. A per-`FETCH` mode remains available as `fault_mode=fetch` for extra experiments.

6.2 Optional enhancements

Table 8. Optional optimizations, each measured on its own.

Enhancement	What we do	Expected effect
Zero-copy send	`sendfile()` from file to socket instead of a 64 KiB `read()` / `send()` loop	Less CPU and no user-space copy per byte
Header coalescing	`TCP_CORK` (or `MSG_MORE`) around each header and its file data	Header and first data share a segment; fewer small packets
Socket buffers	Try explicit `SO_SNDBUF` / `SO_RCVBUF` against Linux autotuning. Setting them turns autotuning off for that socket, and the kernel caps the value at `net.core.wmem_max` / `rmem_max`, which is small by default and needs root to raise. We read the effective size back with `getsockopt()`.	Keep autotuning unless the measurement shows a gain
Client write buffering	1 MiB `setvbuf` buffer per open file; no `fsync()` per file	Fewer system calls on the receive side
Persistent connections	Many `FETCH` batches per connection (already part of 6.1)	No per-batch TCP handshake or slow start

6.3 Socket mode

We use blocking sockets with one thread per connection. At the scale of this project (at most about 64 simultaneous connections) blocking I/O keeps each thread's logic linear and easy to reason about during faults. Non-blocking sockets with `epoll` pay off mainly at thousands of connections, so we record them as the alternative and do not plan to use them.

6.4 How the improvement will be shown

Each enhancement can be switched off by an optional argument (`conns=1`, `sendfile=0`, `sndbuf=0` for autotuning), so we measure an ablation series: Case 2 baseline, then with parallel connections, then with `sendfile()`, then with buffer settings, at p = 0, 0.2 and 0.5. We report the throughput gain over Case 2 and over Part I.

7 Additional features beyond the minimum

These features make the programs safe to leave running during long experiments and make results reproducible. None of them changes the required command-line interface.

Table 9. Additional features.

Feature	What it does	Why
Optional `key=value` arguments	`mode`, `seed`, `ckpt_kb`, `conns`, `max_clients`, `max_retries`, `session_ttl`, `fault_mode`, `sendfile`, `sndbuf`, `log`; every one has a default	Tune experiments without breaking the auto-grader's invocation
Per-connection metrics CSV	Server and client each append one row per connection or attempt (Section 9)	Raw data for all tables and graphs, as the Results folder requires
Leveled logging	`log=0..3` (errors to debug) to stderr. In the multi-threaded servers a logger thread writes CSV rows so workers never block on file I/O; in Part I single-threaded mode the one server thread writes each row after the client's timing ends.	Debug faults without slowing the data path

Table 9 (continued)

Feature	What it does	Why
Graceful shutdown	`SIGINT` / `SIGTERM` set a stop flag through a self-pipe; the accept loop stops, active transfers get 5 s to finish, and CSVs are flushed	No truncated logs or half-written results
Robustness	`SIGPIPE` ignored, `MSG_NOSIGNAL`, 10 s server timeout on reading a request, 1024-byte line limit, `EINTR` retried. The client sets no timeout while waiting for its first reply, because in single-threaded mode a queued client waits for every earlier client to finish.	A misbehaving or silent client cannot crash the server or hold a thread forever
Path safety	Topic keys with `/` or `..` rejected; the client writes only the base name of each file	Neither side can be tricked into touching files outside its directory
Topic extensibility	Topics are discovered by scanning the root at startup, so a new topic is a new folder picked up at the next start. Rescanning while running is deliberately left out, because it could change a topic under an active session.	New topics need no code change
`LIST` command	Returns `TOPIC <key> <file_count> <total_bytes>` lines, then `END`. For debugging only; the client never sends it and grading does not depend on it.	Quick check of what a server offers
Connection cap	`max_clients` (default 128) limits worker threads	Prevents thread exhaustion under a flood of clients
Integrity check	`verify.sh` compares the SHA-256 of every received file with the source	Shows that resuming never corrupts or duplicates data
Clear exit codes	0 success, 1 usage, 2 gave up after retries, 3 topic not found, 4 I/O error	Experiment scripts detect failures automatically

8 Code structure

Each case folder is self-contained and builds with its own Makefile into exactly two executables, `server` and `client`. Shared modules are copied into each folder, so no folder depends on another at build time.

```
FinalSubmission_Group_021_034/
  FinalReport_Group_021_034.pdf
  README.md
  Part1/                  Makefile  common.h/.c  topics.h/.c  proto.h/.c  metrics.h/.c
                          server.c  transfer.c  client.c
  Part2_Case1/            Part1 files + fault.h/.c, retry loop in client.c
  Part2_Case2/            Case 1 files + session.h/.c, checkpoint.h/.c
  Part2_Case2_Enhanced/   Case 2 files + manifest.c, workq.h/.c, parallel client
  Results/
    Part1_SingleThread_throughput.csv / .png
    Part1_MultiThread_throughput.csv / .png
    Part2_Case1_throughput.png  Part2_Case1_overhead.png  (likewise Case2, Case2_Enhanced)
    scripts/  run_part1.sh  run_part2.sh  verify.sh  plot.py
```

8.1 Key modules and functions

Table 10. Modules.

Module	Key functions	Used in
`common.c`	`send_all()`, `lr_init()`, `lr_readline()`, `lr_read_exact()`, `now_ns()`, `parse_opts()`	All
`topics.c`	`topic_map_build(root)`, `topic_lookup(map, key)`, `topic_pos(t, idx, off)`, `topic_stream_size(t, idx, off)`	All servers
`proto.c`	`proto_parse_request()`, `proto_fmt_file_hdr()`, `proto_parse_file_hdr()`	All
`transfer.c`	`send_topic(conn, topic, from_idx, from_off, fault_plan)`, `send_file_range()`	All servers
`server.c`	`main()`, `listen_on(port)`, `accept_loop()`, `serve_client(conn)`, `client_thread(arg)`	All servers
`metrics.c`	`metrics_row()`, logger thread (multi-threaded servers)	All
`fault.c`	`fault_plan_new(p, R, rng)`, `fault_budget(plan)`, `fault_abort(fd)`	Part II servers
`session.c`	`session_create()`, `session_validate_resume()`, `session_update_hw()`, `session_finish()`, `reaper_thread()`	Case 2, Enhanced
`checkpoint.c`	`ckpt_load()`, `ckpt_save_atomic()`, `ckpt_remove()`	Case 2, Enhanced clients
`client.c`	`fetch_topic()`, `recv_files()`, `retry_loop()` (Part II), `worker_main()` (Enhanced)	Clients
`workq.c`	`workq_build(manifest, 1 MiB)`, `workq_take()`, `workq_requeue()`	Enhanced client

8.2 How the variants fit together

Table 11. What each variant adds.

Variant	How it is built
Single vs multi-threaded (Part I)	Both run the same `serve_client()`. In single mode the accept loop calls it directly, so the next client waits in the listen backlog (128), and no other thread is created. In multi mode the accept loop creates a detached thread running `client_thread()`, which calls `serve_client()`. The mode is the optional argument `mode=single` or `mode=multi`; the default is `multi`.
Case 1	Passes a fault plan into `send_topic()`; `send_file_range()` limits each write to `fault_budget()` and calls `fault_abort()` at zero. The client wraps `fetch_topic()` in `retry_loop()`.
Case 2	Adds a `RESUME` branch in `serve_client()` that calls `session_validate_resume()` and then `send_topic()` from the checkpoint. The client saves checkpoints inside `recv_files()`.
Enhanced	Adds the `ATTACH` / `FETCH` loop in `serve_client()` and replaces the single receive loop with N `worker_main()` threads sharing `workq`.

8.3 Build and command-line interface

The Makefile compiles with `gcc -std=c11 -O2 -Wall -Wextra -Werror -pthread`, so any warning fails our own build before submission. Every source file defines `_GNU_SOURCE` before its first include, because `-std=c11` alone hides POSIX and Linux declarations such as `erand48()`, `strcasecmp()` and `sendfile()`, which would otherwise show up as warnings. `make` produces `server` and `client`; `make clean` removes them. No path, port or IP address is hard-coded; every one comes from the command line. Invalid arguments, including a failure probability outside $[0, 1]$, print a usage message and exit with code 1.

Table 12. Command-line interface.

Program	Required invocation	Optional trailing arguments
Part I server	`./server <port> <topics_root_dir>`	`mode=`, `log=`, `max_clients=`
Part II server (all cases)	`./server <port> <topics_root_dir> <failure probability>`	`seed=`, `fault_mode=`, `session_ttl=`, `log=`, `max_clients=`; Enhanced also `sendfile=`, `sndbuf=`
Client (all cases)	`./client <server_ip> <server_port> <topic> <output_dir>`	`max_retries=` (Part II), `ckpt_kb=` (Case 2, Enhanced), `conns=` (Enhanced), `log=`

9 Measurement methodology and experiment plan

Part I is measured by aggregate system throughput: the total bytes sent by the server divided by the span from the first client's request to the last client's completion. This is not an average of per-client rates. Every run is repeated 5 times and reported as mean ± standard deviation.

9.1 What is recorded

The server writes one CSV row per connection, timed with `clock_gettime(CLOCK_MONOTONIC)`: start after the request is parsed, end after the last `send()` returns. Part I throughput is computed from server rows only, so it never mixes clocks from different machines.

```
run_id,case,mode,p,conn_id,client_ip,topic,t_start_ns,t_end_ns,
bytes_sent,bytes_acked,payload_bytes,files,outcome
```

The client writes one row per attempt and one summary row per topic: attempts, bytes received, useful, redundant and control bytes, and completion time from the first `connect()` to `END` (to the last completed file in Case 2 Enhanced). Because `send()` returning means the data reached the server's kernel, not the client, the client-side completion time is reported as a cross-check.

9.2 Metrics

Let B_i be the bytes sent on connection i, with start and end times t_i^{start} and t_i^{end}, over N clients.

$$\Theta_{\text{agg}} = \frac{\sum_{i=1}^{N} B_i}{\max_i t_i^{\text{end}} - \min_i t_i^{\text{start}}} \qquad r_i = \frac{B_i}{t_i^{\text{end}} - t_i^{\text{start}}}$$

Θ_{agg} is the aggregate system throughput. The per-client transfer rate r_i is reported alongside it, to show queueing in single-threaded mode and fairness in multi-threaded mode.

Byte accounting. Every byte on a connection is either payload (file contents) or control (requests, headers, `END` and session lines):

$$B_{\text{total}} = B_{\text{useful}} + B_{\text{redundant}} + B_{\text{control}}$$

- B_{useful} is the topic's `total_bytes`: each file byte delivered once.
- $B_{\text{redundant}}$ is the payload received beyond the useful bytes: bytes discarded at a restart (Case 1) or by truncation to the checkpoint (Case 2).
- B_{control} is excluded from both and reported separately.

With T the client's completion time:

$$\text{goodput} = \frac{B_{\text{useful}}}{T}, \qquad \text{throughput} = \frac{B_{\text{total}}}{T}, \qquad \text{overhead} = \frac{B_{\text{redundant}}}{B_{\text{useful}}}$$

These counts come from the client, which counts the bytes it actually received. The server's `bytes_sent` overstates cut connections, because the RST discards whatever is still in the send buffer. The server therefore also logs `tcpi_bytes_acked` from `getsockopt(TCP_INFO)` just before each abort, and keeps its own counts as a cross-check.

9.3 Experiment matrix

Table 13. Planned experiments.

Experiment	Setup	Varied	Graphs
Part I	1 server node and 2 client nodes; clients split evenly and launched together by one script over ssh	Clients N = 1, 2, 4, 8, 16, 32; single vs multi mode	Aggregate system throughput vs N, both modes on one plot
Part II Case 1	Multi-threaded server, N = 1 and N = 8	p = 0, 0.1, 0.2, 0.3, 0.5, 0.7, 0.9	Throughput vs p; overhead vs p
Part II Case 2	As Case 1	Same p values; checkpoint interval 16, 64, 256 KiB	Same, with Case 1 overlaid
Case 2 Enhanced	As Case 1	Same p values; `conns` = 1, 2, 4, 8; ablation of each enhancement	Throughput and overhead vs p; gain vs `conns`; faults per topic

Controls. Topic data is read once before timing so every run starts with the same warm page cache. Clients on the same node write to separate output directories. The link speed of the lab network is recorded with each run. Part II results at $p = 0$ are compared with Part I multi-threaded results to show the cost of session bookkeeping alone.

10 Design thought process and alternatives

Our guiding rule was to add one mechanism per case, keep everything else the same, and prefer the simplest design whose behaviour we can predict, so that each measured difference has a single, explainable cause.

Table 14. Decisions and alternatives.

Decision	Alternatives considered	Chosen	Reason
Wire format	Fixed binary headers in network byte order; length-prefixed TLV	ASCII header lines plus raw file bytes	Matches the required `GET <topic>`, debuggable with `nc`, header cost well under 1%
Topic map	Hash table; re-reading the directory on every request	Sorted arrays built at startup, `bsearch()`	10 topics; stable file indices for checkpoints; lock-free reads
Multi-threaded model (Part I)	Fixed thread pool with a job queue; separate reader and sender threads per client	Thread per client, detached, capped by `max_clients`	Fewest moving parts at up to 32 clients; each transfer is independent; a pool is kept as a fallback if thread creation cost shows in the results
Separate thematic threads	None; a thread per task type	Accept and worker threads; logger (multi-threaded servers only); reaper (Case 2 and Enhanced)	Keeps disk logging and session expiry off the data path, while Part I single mode stays one thread
Fault decision	Per-chunk Bernoulli trial; timer-based kill; client-side kill	One trial per request, cut point uniform over the response	Case 1 stays finishable, p has a clear meaning, cuts hit every protocol state
Fault unit in Enhanced	One trial per `FETCH`	One trial per connection	Keeps the number of faults comparable with Case 2 at the same p
Fault effect	Graceful `close()` (FIN); `shutdown()`	RST via `SO_LINGER` 0	Looks like a real crash or link failure to the client
Case 1 restart	Keep complete files and skip them	Delete this attempt's files, restart from byte 0	The brief requires no retained metadata in Case 1
Checkpoint authority	Server-side offset from `send()` counts; client ACK messages every few KiB	Client checkpoint on disk, validated against the server's high-water mark	Only the client knows what reached disk; no upstream ACK traffic
Checkpoint comparison	Compare (file index, offset) pairs	Convert to a single payload position	The end of one file and the start of the next are the same point; pairs would wrongly reject valid checkpoints
Topic version hash	Hash of file names, sizes and times carried in every session message	Dropped	The map is fixed for the server's lifetime and sessions live in memory, so the hash could never differ within a session
Checkpoint interval	Per file only; per byte	Every file and every 64 KiB	Bounds redundant bytes per cut below 64 KiB; the interval is swept in experiments
Parallel work split	Static round-robin of files; static equal byte ranges; server-driven pushing	Client-driven queue of about 1 MiB batches	Balances load automatically, and a cut stalls only one batch
Socket mode	Non-blocking sockets with an `epoll` event loop	Blocking sockets with threads	Adequate at tens of connections, far simpler failure handling

Part I

We first considered a thread pool, since it avoids creating a thread per client. At the client counts in the lab (up to 32), thread creation costs microseconds against transfers lasting seconds, so thread-per-client is simpler with no expected penalty. We expect the single-threaded server's aggregate throughput to stay roughly flat as N grows, while the multi-threaded server overlaps disk reads and per-connection TCP ramp-up until the network link saturates. If a single TCP stream already fills the lab link, the two curves will be close, and the difference will show mainly in the per-client rates and waiting times.

Case 1

The main question was what p should mean. A per-chunk probability made large topics practically impossible to finish, so we chose one trial per request with a uniform cut point, which also gives the predictions in Section 4 to check our measurements against.

Case 2

We compared server-held and client-held progress. The server cannot know what arrived, since `send()` only fills the kernel buffer and a RST discards it. ACK messages would fix that but add traffic and a second reader per thread. A client checkpoint validated against the server's high-water mark is correct and cheaper. While checking the design we found that comparing checkpoints as (index, offset) pairs would reject valid checkpoints at file boundaries, and that a high-water mark updated only at file boundaries would reject valid in-file checkpoints; both are fixed by updating the mark after every send and comparing single positions.

Case 2 Enhanced

Static splits suffer when one connection is cut repeatedly and falls behind. Dynamic batches of about 1 MiB let healthy connections absorb the work. The batch size trades scheduling overhead (more `FETCH` round trips) against the amount of work tied to one connection. We also moved the fault decision from each `FETCH` to each connection, so that Case 2 and Case 2 Enhanced face a comparable number of faults at the same p.

References

1. Animals-10 dataset, A. Corrado, Kaggle: kaggle.com/datasets/alessiocorrado99/animals10
2. The Open Group Base Specifications, POSIX.1-2024 (IEEE Std 1003.1-2024): pubs.opengroup.org/onlinepubs/9799919799/
3. LLNL, POSIX Threads Programming tutorial: hpc-tutorials.llnl.gov/posix/
4. Linux manual pages: `socket(7)`, `tcp(7)`, `sendfile(2)`, `clock_gettime(2)`